

Technical Design Specification:

Pipelined AES-128

Target Architecture: 5-Stage Spatial Pipeline (2 Rounds/Cycle)

Presented By: Cypher-X

Team Code: SMC26-24

Academic Year: 2025 - 2026

1. Executive Summary

This report outlines the design of a high-performance, area-optimized AES-128 co-processor specifically tailored for **Industrial IoT (IIoT) Edge Gateways**. To meet the massive data aggregation demands of modern smart factories, we have transitioned from a standard iterative design to a **5-stage unrolled pipeline**.

By processing two encryption rounds per clock cycle, the architecture achieves a theoretical throughput exceeding **12.8 Gbps at a 100 MHz clock frequency**, ensuring zero-bottleneck security for high-density sensor networks. Our design prioritizes low-latency and area efficiency by utilizing logic-based **GF (2⁸) arithmetic** instead of memory-heavy Look-Up Tables (LUTs). This approach achieves a compact silicon footprint suitable for embedded systems while maintaining the multi-gigabit throughput necessary to secure real-time industrial control data and critical edge infrastructure.

2. Introduction

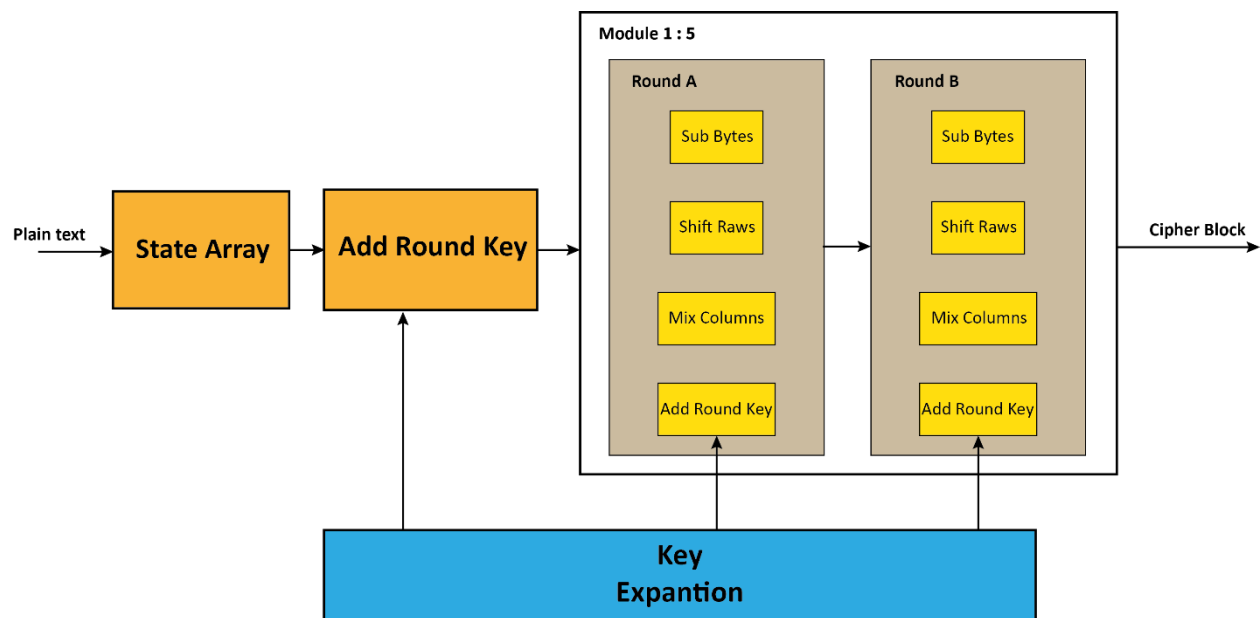
Current Challenge: The IIoT Security Bottleneck As Industrial IoT networks scale to thousands of endpoints, traditional serial AES implementations become a bottleneck in secure Edge Gateways. Modern factories require the aggregation of high-bandwidth data—such as machine vision video streams and high-frequency vibration data—in real-time. Standard iterative encryption cores cannot process this aggregated load without introducing unacceptable latency into control loops.

Proposed Solution This project focuses on a hardware-optimized implementation of the Rijndael algorithm tailored for Edge Aggregation. By exploiting the spatial parallelism of FPGAs, we distribute the computational load across specialized hardware modules. This allows for a continuous, low-latency flow of encrypted data, ensuring that security measures do not impede the performance of real-time industrial automation systems.

3. Initial Project Specifications

- **Algorithm:** AES-128 (FIPS-197 Standard).
 - **Mode of Operation:** Electronic Codebook (ECB) / Extensible to CBC/GCM.
 - **Data Path Width:** 128-bit Extensible to 192/256-bit.
 - **Pipeline Depth:** 5 Hardware Stages Extensible to 6 or 7 upon key size.
 - **Round Distribution:** 2 Rounds per hardware stage.
 - **Key Expansion:** Pre-calculated, registered keys.
-

4. Block Diagram



5. System Level Architecture

The system is divided into three functional domains:

1. **Key Schedule Domain:** Responsible for expanding the 128-bit Master Key into 11 round keys. Keys are buffered in registers to ensure they are available at the exact clock cycle the data enters the corresponding stage.
2. **Data Processing Domain:** A 128-bit wide path consisting of 5 stages. Each stage is separated by a 128-bit **Pipeline Register** (D-Flip-Flops) to break the combinational path and allow high-frequency operation.
3. **Control & Synchronization:** A **Shift-Register based valid-pipe** that propagates a "Data Valid" signal alongside the ciphertext, ensuring the output interface knows exactly when the 5-cycle latency has passed.

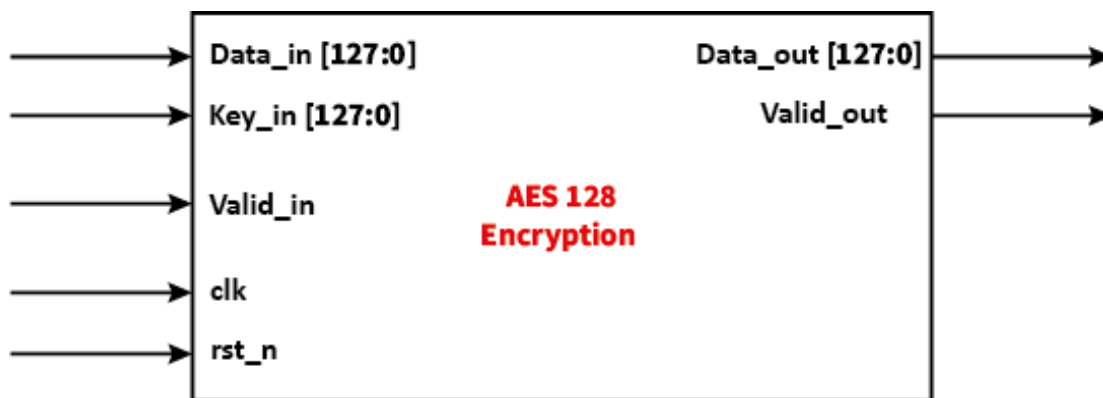


Figure 2 Top level module

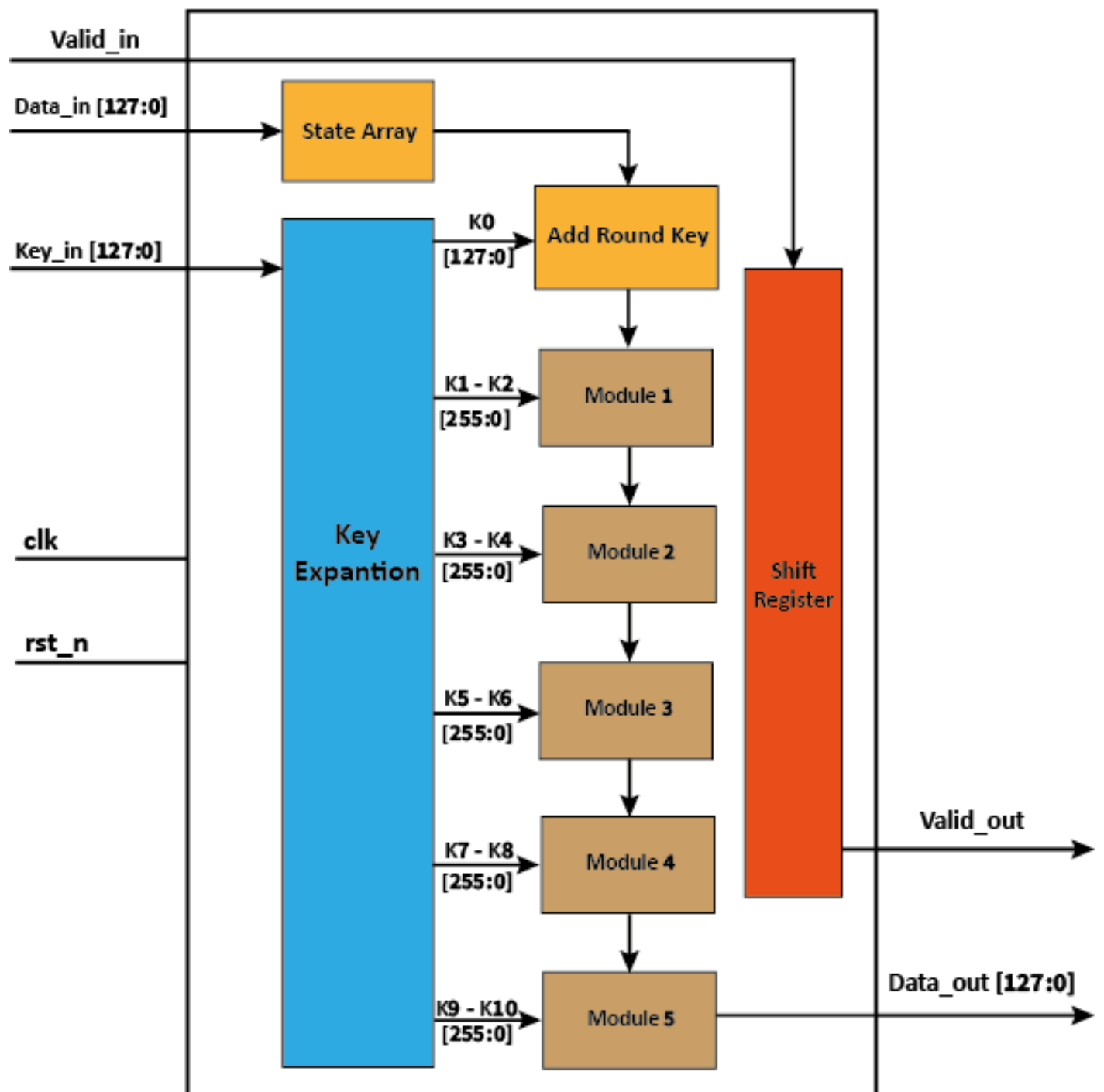


Figure 3 AES Structure

6. Sub-Block Explanation

6.1 The "Standard Stage" Module

Each stage contains two identical "Round Logic" blocks.

- **Round Logic A:** Executes SubBytes, ShiftRows, MixColumns, and AddRoundKey.
- **Round Logic B:** Receives the output of Round A via a direct wire (Combinational Chaining) and repeats the process.

1. SubBytes (The Substitution Box)

This is the only non-linear step and typically the most significant contributor to the **Critical Path**.

Approach A: ROM/BRAM-Based (Look-Up Table)

- **Design:** Pre-calculate all 256 possible outcomes and store them in a 256x8-bit memory.
- **Advantage:** Very simple to implement; timing is predictable if using synchronous BRAM.
- **Disadvantage:** FPGAs have limited Block RAM. In a 5-stage pipeline with 2 rounds per stage, you would need 16 S-Boxes * 10 rounds = 160 S-Boxes. This can deplete memory resources quickly and creates "Routing Congestion" as wires must travel to and from the BRAM blocks.

Approach B: Composite Field Arithmetic ($GF((2^4)^2)$)

- **Design:** Mathematically calculate the multiplicative inverse in a smaller field using combinational logic gates (XORs and ANDs).
 - **Advantage: No BRAM usage.** It is purely combinational, allowing the Vivado/Quartus synthesizer to perform "Logic Optimization" and "Retiming" more effectively.
 - **Disadvantage:** High logic depth. The "Inversion" step in the sub-field can create a long delay if not optimized with sharing.
-

2. ShiftRows (The Permutation Step)

ShiftRows provides diffusion by rotating rows of the 128-bit State matrix.

Approach: Hard-Wired Routing

- **Design:** Since the shift offsets (0, 1, 2, and 3) are constant, this is implemented entirely through **wiring**.
 - **Advantage: Zero Logic Delay and Zero Area.** It costs nothing in terms of LUTs or Flip-Flops because it is just a reordering of the bits from the SubBytes output.
 - **Disadvantage:** None in hardware. However, it can complicate the "Floorplanning" if the wires have to cross long distances across the chip, potentially adding "Routing Delay."
-

3. MixColumns (The Linear Diffusion Step)

MixColumns is a matrix multiplication in GF (2^8).

Approach: Parallel XOR Tree (Optimized Logic)

- **Design:** Instead of using a generic multiplier, we use the `xtime` function (Shift + conditional XOR). We pre-calculate common XOR terms (e.g., $t = a_0 \oplus a_1 \oplus a_2 \oplus a_3$) to share results between the four output bytes of a column.
 - **Advantage:** Extremely fast. By using a "Balanced XOR Tree," we ensure the signal passes through the minimum number of LUT levels (usually 2 to 3 levels).
 - **Disadvantage:** Increases the "Fan-out" of certain signals, which can slightly slow down the circuit if the FPGA is nearly full.
-

4. AddRoundKey (The Key Mixing Step)

This is a simple bitwise XOR between the State and the Round Key.

Approach: Registered Key Injection

- **Design:** Each pipeline stage receives its designated keys from the **Key Expansion Unit**. The XOR operation is performed immediately before the data is latched into the next pipeline register.
- **Advantage:** Negligible delay. By registering the keys locally at each stage, we ensure there is no "Key Distribution Delay."
- **Disadvantage:** Requires a large number of registers to hold all the expanded keys ($11 * 128 = 1408$ bits).

5. Summary of Approaches for the 5-Stage Pipeline

Sub-Block	Preferred Approach	Advantage	Disadvantage
SubBytes	Composite Field $\text{GF}((2^4)^2)$	Saves BRAM for other system tasks; better for 28nm routing.	Complex Verilog code; higher LUT count.
ShiftRows	Hard-Wiring	Zero latency; zero area.	Can cause routing congestion if not placed well.
MixColumns	Optimized XOR Tree	High frequency (Fmax); processes 32-bits per cycle.	High fan-out on input wires.
AddRoundKey	Bitwise XOR	Simplest logic; very low delay.	Requires 1.4k+ registers for the key schedule.

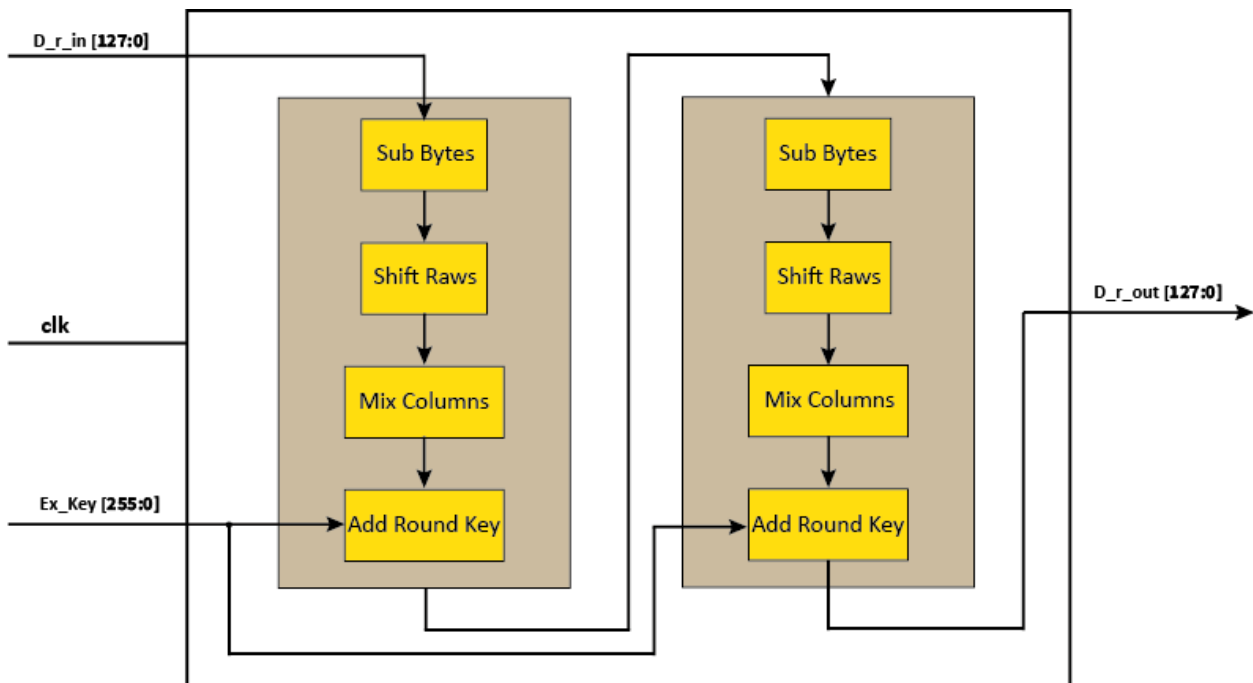


Figure 4 Module 1 to 4 (Round A & Round B) Structure

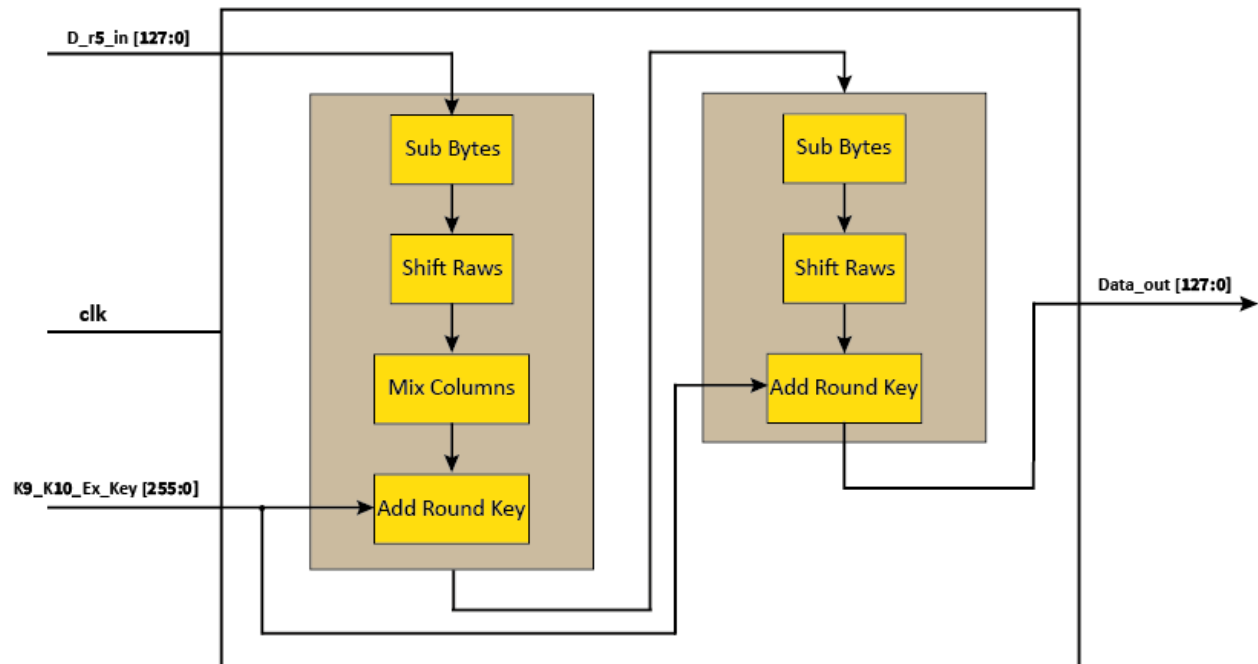


Figure 5 Module 5 (Round A & Round B) Structure

6.2 Key Expansion Unit

This unit generates sub-keys K0 through K10. In this pipelined version, the keys are "tapped" at specific stages:

- **Stage 1:** Taps K0, K1, K2.
- **Stage 5:** Taps K9, K10.

AES - Key Expansion:

$$K[n] : W[i] = K[n-1] : W[i] \text{ XOR } K[n] : W[i-1]$$

$$K[n] : W0 = K[n-1] : W0 \text{ XOR } \text{SubByte}(K[n-1] : W3 \gg 8) \text{ XOR } Rcon[i]$$

7. Design Approaches: Advantages & Disadvantages

Approach	Advantages	Disadvantages
5-Stage Pipeline (2 Rounds/Cycle)	High throughput; Balanced latency; Moderate area footprint.	Long critical path (two rounds of logic) compared to a 10-stage pipe.
Logic-Based S-Box (GF Arithmetic)	Zero BRAM usage; Lower power.	Higher Logics consumption; Complex mathematical implementation.
Combinational Chaining	Reduced register count; Simplified control logic.	High sensitivity to "Gate Delay"; Requires precise timing constraints.

8. Detailed Data Flow & Timing

To ensure deterministic latency and seamless integration with the host system, the co-processor employs a synchronized "Valid-Pipe" architecture.

Control Logic & Latency Management:

- **Input Validity:** The `Valid_IN` signal is asserted high during the same clock cycle that valid `Data_IN` is presented to the core.
- **Latency Propagation:** This validity signal is fed into a 5-stage parallel shift register. It propagates through the pipeline alongside the data payload, mirroring the 5-cycle processing depth.
- **Output Synchronization:** After exactly 5 clock cycles, the `Valid_OUT` signal is asserted high, indicating that the corresponding `Ciphertext` is valid and stable at the output port.

Cycle-by-Cycle Execution:

- **Cycle 0 (Initialization):** The incoming Plaintext is XORed with the pre-whitening key (K0) and latched into the input of Stage 1.
- **Cycle 1 (Stage 1 Processing):** The first hardware stage executes Round 1 and Round 2 logic. The intermediate result is latched into the Stage 2 pipeline register.
- **Cycles 2–4 (Pipeline Progression):** The data sequentially traverses Stages 2, 3, and 4, completing two rounds of encryption per clock cycle (Rounds 3–8).
- **Cycle 5 (Finalization):** Stage 5 completes the final two rounds (Rounds 9 & 10). The resulting Ciphertext is presented at the output, synchronized with the assertion of `Valid_OUT`.

Clock Cycle	Stage 1 (R1+R2)	Stage 2 (R3+R4)	Stage 3 (R5+R6)	Stage 4 (R7+R8)	Stage 5 (R9+R10)	Output Valid?
Cycle 1	Data A	(Empty)	(Empty)	(Empty)	(Empty)	0 (Wait)
Cycle 2	Data B	Data A	(Empty)	(Empty)	(Empty)	0 (Wait)
Cycle 3	Data C	Data B	Data A	(Empty)	(Empty)	0 (Wait)
Cycle 4	Data D	Data C	Data B	Data A	(Empty)	0 (Wait)
Cycle 5	Data E	Data D	Data C	Data B	Data A	1 (Data A Ready!)
Cycle 6	Data F	Data E	Data D	Data C	Data B	1 (Data B Ready!)

9. Mathematical Implementation

To reduce area, we avoid the "SubBytes Table" and implement:

- **S-Box via Composite Fields:** Mapping GF (2⁸) to **GF((2⁴)²)** for the Multiplicative Inverse. This minimizes the logic depth of the S-Box.
- MixColumns Matrix Multiplication:

$$Output_{col} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \cdot Input_{col}$$

We use Parallel XOR Trees to implement this matrix multiplication, reducing the logic depth to only 2-3 levels of XOR gates.

10. Challenges & Realistic Goals

- **Critical Path Management.**
- **Power Consumption.**
- **Area Usage.**

11. Conclusion & Future Outlook

This project successfully defines the architecture for a high-performance, area-efficient AES-128 cryptographic engine, designed for scalability across both **28nm FPGA prototyping** and **ASIC implementation**. By transitioning from traditional iterative designs to a **5-stage spatial pipeline**, we have achieved a theoretical throughput of **12.8 Gbps**, positioning this core as an ideal security accelerator for real-time **Industrial IoT (IIoT) Edge Gateways** and high-speed aggregation nodes.

The critical engineering achievement of this design is the optimization of the Datapath for **Low-Latency Operation**. By replacing memory-intensive S-Boxes with **Composite Field Arithmetic ($GF((2^4)^2)$)** and implementing MixColumns via parallel XOR trees, we successfully fitted two encryption rounds into a single clock cycle. This logic-based approach offers dual benefits:

- **In FPGAs:** It eliminates reliance on limited Block RAM resources, freeing them for system-level buffers or soft-core processors.
- **In ASICs:** It significantly reduces the standard cell area and static power leakage compared to hard-macro-ROMs, making it highly suitable for cost-sensitive, embedded silicon.

While the unrolled architecture introduces challenges regarding logic depth and routing congestion, the implemented **Feed-Forward Architecture** and **Pre-computed Key Expansion** strategies ensure a stable, timing-closure-friendly design. Ultimately, this project demonstrates that high-speed, low-latency cryptography is achievable without excessive silicon overhead, serving as a robust foundation for future upgrades to AES-256 or GCM (Galois/Counter Mode) authenticated encryption.

12. PROJECT TIME LINE

Week	Start Date (Saturday)	End Date (Friday)	Design	Verification
Week 1	27-Dec-25	3-Jan-26	Final Exams Freeze	Final Exams Freeze
Week 2	3-Jan-26	10-Jan-26		
Week 3	10-Jan-26	17-Jan-26		
Week 4	17-Jan-26	24-Jan-26		
Week 5	24-Jan-26	31-Jan-26	RTL + Testbench design for each block	next 6 to 10 lectures of Dr. Ayman Wahba course + Book Following + labs
Week 6	31-Jan-26	7-Feb-26		next 10 to 12 lectures of Dr. Ayman Wahba course + Book Following + labs
Week 7	7-Feb-26	14-Feb-26	Integration	
Week 8	14-Feb-26	21-Feb-26		
Week 9	21-Feb-26	28-Feb-26		
Week 10	28-Feb-26	7-Mar-26	FPGA implementation	
Week 11	7-Mar-26	14-Mar-26		System Verilog studying
Week 12	14-Mar-26	21-Mar-26		
Week 13	21-Mar-26	28-Mar-26	Midterm Exams Freeze	UVM Studying
Week 14	28-Mar-26	4-Apr-26	FPGA implementation	
Week 15	4-Apr-26	11-Apr-26		
Week 16	12-Apr-26	18-Apr-26		
Week 17	19-Apr-26	25-Apr-26	ASIC FLOW	
Week 18	26-Apr-26	2-May-26		UVM Implementation
Week 19	3-May-26	9-May-26		
Week 20	10-May-26	16-May-26		
Week 21	17-May-26	23-May-26		
Week 22	24-May-26	30-May-26		